

C2SafeRust-LLM

Module 4: Interprocedural Analysis -- Line-by-Line Explanation

For Beginners | Mini Project S3

Module 4: Interprocedural Analysis

Line-by-Line Code Explanation for Beginners

Files: `mod.rs`, `call_graph.rs`, `alias_analysis.rs`, `ownership_summary.rs`, `union_tracking.rs`, `data_dep_detector.rs`

What does Module 4 do? Module 3 looked at each function IN ISOLATION. Module 4 looks at how functions INTERACT WITH EACH OTHER across the whole program. It detects and resolves the 6 C Memory Idioms Taxonomy — complex C patterns where the correct Rust ownership type can only be determined by looking at multiple functions together. Inputs: `TranslationUnitAST` (Module 2) + `HashMap` of `SemanticModels` (Module 3 D1) Output: `InterproceduralSummaries` (D2) — the whole-program analysis result

The 6 C Memory Idioms This Module Resolves:

```
Idiom 1: Aliased &mut parameters -> sub-module: alias_analysis.rs
Idiom 2: T** consumes ownership -> sub-module: ownership_summary.rs
Idiom 3: T** creates ownership -> sub-module: ownership_summary.rs
Idiom 4: Overlapping pointer slices -> sub-module: alias_analysis.rs
Idiom 5: Cross-function union variants -> sub-module: union_tracking.rs
Idiom 6: Data-dependent ownership -> sub-module: data_dep_detector.rs
```

File 1: `src/module4_interprocedural/mod.rs`

(The Orchestrator — Runs All 5 Sub-Analyses in Sequence)

Lines 1-11: Module declarations and re-exports

```

pub mod call_graph;
pub mod alias_analysis;
pub mod ownership_summary;
pub mod union_tracking;
pub mod data_dep_detector;

pub use call_graph::{build_call_graph, CallGraphResult};
pub use alias_analysis::analyze_interprocedural_aliasing;
pub use ownership_summary::generate_ownership_transfer_summaries;
pub use union_tracking::track_cross_function_union_variants;
pub use data_dep_detector::detect_data_dependent_ownership;

```

- Five sub-modules declared with `pub mod`.
- Each main function re-exported at the module level for convenient access.

Lines 13-16: Cross-module imports

```

use std::collections::HashMap;
use crate::module2_ast::TranslationUnitAST;
use crate::data_stores::semantic_models::SemanticModel;
use crate::data_stores::interprocedural_summaries::InterproceduralSummaries;

```

Import	Role
<code>`HashMap`</code>	<code>`semantic_models`</code> is <code>`HashMap`</code> .
<code>`TranslationUnitAST`</code>	The whole-program AST from Module 2.
<code>`SemanticModel`</code>	The D1 data from Module 3.
<code>`InterproceduralSummaries`</code>	The D2 data store we're filling in.

Lines 18-42: The Main Entry Point `run_interprocedural_analysis()`

```

pub fn run_interprocedural_analysis(
    tu_ast: &TranslationUnitAST,
    semantic_models: &HashMap<String, SemanticModel>,
) -> InterproceduralSummaries {
    let mut summaries = InterproceduralSummaries::new();

    // Step 4.1: Build Call Graph
    let call_graph_res = build_call_graph(tu_ast);
    summaries.call_graph = call_graph_res.graph;

    // Step 4.2: Alias Analysis (Idiom 1 & 4)
    summaries.alias_verdicts = analyze_interprocedural_aliasing(tu_ast);

    // Step 4.3: Ownership Transfer Summaries (Idiom 2 & 3)
    summaries.ownership_transfer_summaries =
        generate_ownership_transfer_summaries(tu_ast, semantic_models);

    // Step 4.4: Cross-Function Union Tracking (Idiom 5)
    summaries.cross_function_union_resolutions =
        track_cross_function_union_variants(tu_ast, semantic_models);

    // Step 4.5: Data-Dependent Ownership (Idiom 6)
    summaries.data_dependent_ownership_flags =
        detect_data_dependent_ownership(tu_ast, semantic_models);

    summaries
}

```

Design Decision	Explanation
<code>`let mut summaries = InterproceduralSummaries::new()`</code>	Start with an empty D2 store.
Each sub-call sets one field	<code>`summaries.alias_verdicts = ...`</code> fills the <code>`alias_verdicts`</code> field directly.
Both D1 and D2 used	Steps 4.3–4.5 need <code>`semantic_models`</code> (D1) to cross-reference what Module 3 found. Step 4.1 and 4.2 only need the AST.
<code>`summaries`</code> returned by value	The compiled <code>`InterproceduralSummaries`</code> is moved to the caller — no cloning needed.

File 2: `src/module4_interprocedural/call_graph.rs`

(Building the Whole-Program Function Call Graph)

What is a Call Graph? A call graph is a directed graph where: - **NODES** = functions (e.g., `main`, `swap`, `foo`) - **EDGES** = function calls (e.g., `main` calls `swap` on line 15) It lets us answer questions like: - "Which functions does `main` call?" - "Which functions call `swap`?" - "Is there any function that is never called?" We use the `petgraph` library which provides efficient graph data structures.

Lines 1-4: Imports

```
use std::collections::HashMap;
use petgraph::graph::{Graph, NodeIndex};
use crate::module2_ast::TranslationUnitAST;
use crate::data_stores::interprocedural_summaries::CallSite;
```

Import	Why needed
<code>`HashMap`</code>	<code>`node_indices`</code> maps function names to their graph node IDs.
<code>`petgraph::graph::Graph`</code>	The directed graph data structure.
<code>`petgraph::graph::NodeIndex`</code>	A unique ID for each node in the graph (returned when adding a node).
<code>`CallSite`</code>	The data stored on each EDGE of the graph (caller, callee, line).

Lines 6-9: The Result Struct

```
pub struct CallGraphResult {
    pub graph: Graph<String, CallSite>,
    pub node_indices: HashMap<String, NodeIndex>,
}
```

Field	Type	What it stores
<code>`graph`</code>	<code>`Graph`</code>	The actual graph. Node data = function name (String). Edge data = <code>`CallSite`</code> .
<code>`node_indices`</code>	<code>`HashMap`</code>	Maps <code>`"swap"`</code> → its <code>`NodeIndex`</code> in the graph. Needed to add edges between nodes efficiently.

Understanding `Graph<String, CallSite>`:

- `Graph<N, E>` is petgraph's directed graph type.
- `N` = node data type = `String` (stores the function name at each node).
- `E` = edge data type = `CallSite` (stores caller/callee/line at each edge).
- When you call `graph.add_node("swap".to_string())`, it returns a `NodeIndex` — like a handle to that node.

Lines 11-43: The `build_call_graph()` Function

```
pub fn build_call_graph(tu_ast: &TranslationUnitAST) -> CallGraphResult {
    let mut graph = Graph:::<String, CallSite>::new();
    let mut node_indices = HashMap::new();

    // Phase 1: Add all function nodes
    for func in &tu_ast.functions {
        let node_idx = graph.add_node(func.name.clone());
        node_indices.insert(func.name.clone(), node_idx);
    }

    // Phase 2: Add edges for each call site
    for func in &tu_ast.functions {
        if let Some(&caller_idx) = node_indices.get(&func.name) {
            for call in &func.call_sites {
                if let Some(&callee_idx) = node_indices.get(&call.callee) {
                    graph.add_edge(
                        caller_idx,
                        callee_idx,
                        CallSite {
                            caller: func.name.clone(),
                            callee: call.callee.clone(),
                            line: call.line,
                        },
                    );
                }
            }
        }
    }

    CallGraphResult { graph, node_indices }
}
```

Phase 1 — Adding Nodes:

Code	Explanation
<code>`Graph:::new()`</code>	Turbofish syntax specifying the type parameters for the new graph.
<code>`graph.add_node(func.name.clone())`</code>	Registers the function as a node. Returns a <code>`NodeIndex`</code> (a numeric ID).
<code>`node_indices.insert(func.name.clone(), node_idx)`</code>	Store <code>`"swap" -> NodeIndex(1)`</code> so we can look up node IDs by name later.

Why two passes (first nodes, then edges)? When adding an edge from `main` to `swap`, we need BOTH node indices. If we tried to add edges as we found nodes, `swap` might not have been registered yet when we encounter the call in `main`. Two-pass ensures all nodes exist before we add edges.

Phase 2 — Adding Edges:

Code	Explanation
<code>`if let Some(&caller;_idx) = node_indices.get(&func.name;)`</code>	Look up the caller's <code>`NodeIndex`</code> . The <code>`&`</code> before <code>`caller_idx`</code> dereferences the <code>`&NodeIndex;`</code> returned by <code>`get()`</code> . <code>`if let Some`</code> safely unwraps.
<code>`for call in &func.call;_sites`</code>	Loop over every <code>`CallSiteAST`</code> in the function's body.
<code>`if let Some(&callee;_idx) = node_indices.get(&call.callee;)`</code>	Look up the callee's <code>`NodeIndex`</code> . If the callee isn't in our program (e.g., an external library function), <code>`get()`</code> returns <code>`None`</code> and we skip it.
<code>`graph.add_edge(caller_idx, callee_idx, CallSite { ... })`</code>	Add a directed edge FROM caller TO callee, carrying the <code>`CallSite`</code> data.

The Unit Test

```
#[test]
fn test_call_graph_construction() {
    // Two functions: main calls foo
    let result = build_call_graph(&tu_ast);
    assert_eq!(result.graph.node_count(), 2); // 2 nodes: main and foo
    assert_eq!(result.graph.edge_count(), 1); // 1 edge: main -> foo
}
```

`node_count()` and `edge_count()` are petgraph built-in methods that count nodes and edges.

File 3: src/module4_interprocedural/alias_analysis.rs

(Idiom 1: Aliased &mut Parameters + Idiom 4: Overlapping Slices)

What is aliasing? Two pointers "alias" when they point to the SAME memory location at the same time. **Why is this a problem in Rust?** Rust's borrow checker FORBIDS having two mutable references to the same memory simultaneously. If C code passes the same variable as two different pointer arguments, the translated Rust code would be rejected by the compiler. Example (Idiom 1): C: `foo(&x, &x)` -- both args point to `x` (aliased) Rust: `foo(&mut x, &mut x)` -- COMPILE ERROR! Two mut refs to same location! Example (Idiom 4): C: `combine(buf + 2, buf + 3, 5)` -- overlapping ranges in same array Rust: `combine(&buf[2..7], &buf[3..8])` -- overlapping slices FORBIDDEN!

Lines 7-39: `analyze_interprocedural_aliasing()` — The Main Function

```

pub fn analyze_interprocedural_aliasing(
    tu_ast: &TranslationUnitAST,
) -> HashMap<String, Vec<AliasingVerdict>> {
    let mut verdicts = HashMap::new();

    for func in &tu_ast.functions {
        for call in &func.call_sites {
            let mut call_verdicts = Vec::new();
            let args = &call.arguments;

            // Check every PAIR of arguments at this call site
            for i in 0..args.len() {
                for j in (i + 1)..args.len() {
                    let arg1 = clean_arg(&args[i]);
                    let arg2 = clean_arg(&args[j]);
                    let aliases = check_if_args_alias(&arg1, &arg2);
                    call_verdicts.push(AliasingVerdict {
                        arg1: args[i].clone(),
                        arg2: args[j].clone(),
                        aliases,
                    });
                }
            }

            let call_key = format!("{}", func.name, call.callee, call.line);
            verdicts.insert(call_key, call_verdicts);
        }
    }
    verdicts
}

```

Code	Explanation
<code>`for i in 0..args.len()`</code>	Outer loop over argument indices.
<code>`for j in (i + 1)..args.len()`</code>	Inner loop starting from <code>`i+1`</code> — ensures we check every UNIQUE PAIR without duplicates. E.g., for args <code>`[a, b, c]`</code> we check pairs <code>`(a,b)`</code> , <code>`(a,c)`</code> , <code>`(b,c)`</code> .
<code>`clean_arg(&args[i])`</code>	Strips <code>`&`</code> and <code>`*`</code> prefixes from argument strings (e.g., <code>`"&x;"`</code> → <code>`"x"`</code>).
<code>`check_if_args_alias(&arg1, &arg2)`</code>	Checks if the two cleaned arguments alias.
<code>`format!("{}", func.name, call.callee, call.line)`</code>	Unique key like <code>`"main:foo:5"`</code> — identifies each call site precisely.

Helper: `clean_arg()` — *Normalize Argument Strings*

```
fn clean_arg(arg: &str) -> String {
    arg.trim_start_matches('&')
      .trim_start_matches('*')
      .trim()
      .to_string()
}
```

Input	Output	Explanation
`"&x;"`	`"x"`	Remove the address-of operator
`"*ptr"`	`"ptr"`	Remove the dereference operator
`"buf + 2"`	`"buf + 2"`	Pointer arithmetic — kept as-is (no & or *)

`trim_start_matches` removes prefix characters. It's called twice — once for `&`, once for `*`. Note: only removes from the START, not the middle. `"&x"` → `"x"`, not `"x"` + removes `&` everywhere.

Helper: `check_if_args_alias()` — The Alias Detection Logic

```
fn check_if_args_alias(arg1: &str, arg2: &str) -> bool {
    // Rule 1: Direct variable alias (e.g. &x and &x)
    if arg1 == arg2 {
        return true;
    }

    // Rule 2: Idiom 4 – Pointer arithmetic slice overlap
    let base1 = get_pointer_base(arg1);
    let base2 = get_pointer_base(arg2);
    if base1 == base2 && !base1.is_empty() {
        return true; // Overlapping slice range detected!
    }

    false
}
```

Rule	C Pattern	Example	Detection
Rule 1 (Idiom 1)	Same variable passed twice	<code>foo(&x;, &x;)</code>	After cleaning: both are <code>"x"</code> → <code>arg1 == arg2</code> → <code>true</code>
Rule 2 (Idiom 4)	Same base pointer with arithmetic	<code>combine(buf+2, buf+3, 5)</code>	<code>get_pointer_base("buf + 2")</code> = <code>"buf"</code> , <code>get_pointer_base("buf + 3")</code> = <code>"buf"</code> → same base → <code>true</code>

Helper: `get_pointer_base()` — Extract Base Array from Pointer Arithmetic

```
fn get_pointer_base(arg: &str) -> String {
    if let Some(idx) = arg.find('+') {
        arg[..idx].trim().to_string()
    } else if let Some(idx) = arg.find('-') {
        arg[..idx].trim().to_string()
    } else {
        String::new()
    }
}
```

Input	Output	Explanation
`"buf + 2"`	`"buf"`	Found `+` at index 4. Take `arg[..4]` = `"buf "`, trim → `"buf"`.
`"buf + 3"`	`"buf"`	Same base — matches!
`"x"`	``	No `+` or `-`. Returns empty string.

If the base is empty, the `!base1.is_empty()` check prevents false positives (two unrelated non-arithmetic args would both have empty base → but we guard against this).

File 4: [src/module4_interprocedural/ownership_summary.rs](#)

(Idiom 2: T Consumes Ownership + Idiom 3: T Creates Ownership)

The Double-Pointer Ownership Problem: In C, passing `int**` (pointer-to-pointer) is used for TWO very different purposes: **Idiom 2 (ConsumesOwnership):** `void free_buffer(int buf) { free(buf); // frees the heap memory the caller owns buf = NULL; } --> In Rust: parameter should be Box<T> (callee takes ownership and drops it) Idiom 3 (CreatesOwnership): void alloc_buffer(int out) { out = malloc(100); // allocates NEW heap memory for the caller } --> In Rust: return a Box<T> or use out-parameter with Box<T> Module 4.3 detects which pattern each T* parameter follows.`

Lines 8-48: `generate_ownership_transfer_summaries()`

```

pub fn generate_ownership_transfer_summaries(
    tu_ast: &TranslationUnitAST,
    semantic_models: &HashMap<String, SemanticModel>,
) -> HashMap<String, OwnershipTransferSummary> {
    let mut summaries = HashMap::new();

    for func in &tu_ast.functions {
        if let Some(model) = semantic_models.get(&func.name) {
            for param in &func.parameters {
                if let Some(PointerDepthClassification::MultiLevel { depth: _ }) =
                    model.pointer_depth_classifications.get(&param.name)
                {
                    let summary_key = format!("{:?}", func.name, param.name);

                    // Check Idiom 2: Callee FREES through T**
                    if func.body_code.contains(&format!("free(*{})", param.name))
                        || func.body_code.contains(&format!("free( *{})", param.name))
                    {
                        summaries.insert(summary_key, OwnershipTransferSummary {
                            param_name: param.name.clone(),
                            transfer_type: OwnershipTransferType::ConsumesOwnership,
                        });
                    }
                    // Check Idiom 3: Callee ALLOCATES through T**
                    else if (func.body_code.contains(&format!("*{} = malloc", param.name))
                        || func.body_code.contains(&format!("*{} = (", param.name)))
                        && func.body_code.contains("malloc")
                    {
                        summaries.insert(summary_key, OwnershipTransferSummary {
                            param_name: param.name.clone(),
                            transfer_type: OwnershipTransferType::CreatesOwnership,
                        });
                    }
                }
            }
        }
    }
    summaries
}

```

Step	Code	Explanation
<code>`semantic_models.get(&func.name);`</code>	Look up D1 model	Get Module 3's SemanticModel for this function. Returns <code>`Option<&SemanticModel>`</code> .
<code>`if let Some(model)`</code>	Guard	Only proceed if we have a model (should always exist, but safe practice).
<code>`model.pointer_depth_classifications.get(&param.name);`</code>	Check if parameter is multi-level	Only T** or deeper parameters can have ownership transfer.
<code>`if let Some(PointerDepthClassification::MultiLevel { depth: _ })`</code>	Pattern match	Confirms the parameter IS multi-level. Skip single-level and non-pointer parameters.

Step	Code	Explanation
<code>`format!("{}", func.name, param.name)`</code>	Unique key	<code>`free_buffer:buf`</code> — identifies this specific function-parameter combination.

Idiom 2 detection pattern strings:

```
param.name = "buf"
format!("free({})", "buf") = "free(*buf)"
format!("free( {})", "buf") = "free( *buf)" // with space variant
```

Both variants cover different coding styles (`free(*buf)` vs `free( *buf)`).

Idiom 3 detection:

```
format!("{}", "out") = "out = malloc"
format!("{}", "out") = "out = (" // cast: *out = (int*)malloc(...)
```

Both cover direct malloc and cast-malloc assignments. The second `&& func.body_code.contains("malloc")` ensures the cast version is actually a malloc call, not some other assignment.

Unit Tests

Test 1 — Idiom 2 (ConsumesOwnership):

```
// Function body: "free(*buf); *buf = NULL;"
// Parameter: buf (int**, depth=2)
// Expected: ConsumesOwnership
assert!(matches!(summaries["free_buffer:buf"].transfer_type,
OwnershipTransferType::ConsumesOwnership));
```

Test 2 — Idiom 3 (CreatesOwnership):

```
// Function body: "*out = malloc(100);"
// Parameter: out (int**, depth=2)
// Expected: CreatesOwnership
assert!(matches!(summaries["alloc_buffer:out"].transfer_type,
OwnershipTransferType::CreatesOwnership));
```

File 5: src/module4_interprocedural/union_tracking.rs

(Idiom 5: Cross-Function Union Variant Tracking)

The Cross-Function Union Problem: Module 3 could only detect union variant usage *WITHIN* a single function. But what if function A sets the union variant, and function B reads it? Example: `void set_val(union Data u) { u->i = 10; }` // sets variant "i" `void read_val(union Data u) { ... }` // reads u -- which variant? Module 3 marks `read_val` as having an UNRESOLVED union case. Module 4.4 performs "reaching definitions"

analysis: look in OTHER functions to find which variant they SET for this union.

Lines 7-45: track_cross_function_union_variants()

```
pub fn track_cross_function_union_variants(
    tu_ast: &TranslationUnitAST,
    semantic_models: &HashMap<String, SemanticModel>,
) -> HashMap<String, String> {
    let mut resolved_cross_variants = HashMap::new();

    // For each function with UNRESOLVED union cases (from Module 3)
    for (func_name, model) in semantic_models {
        for unresolved in &model.unresolved_union_cases {

            let mut found_variant = None;

            // Search ALL OTHER functions for a resolved transform of this union
            for other_func in &tu_ast.functions {
                if other_func.name != *func_name {
                    if let Some(other_model) = semantic_models.get(&other_func.name) {
                        for transform in &other_model.local_union_transforms {
                            if transform.union_name == unresolved.union_name {
                                if let Some(ref variant) = transform.local_variant_resolved {
                                    found_variant = Some(variant.clone());
                                    break;
                                }
                            }
                        }
                    }
                }
            }

            if found_variant.is_some() { break; }
        }

        if let Some(variant) = found_variant {
            let key = format!("{:}:{:}", func_name, unresolved.union_name);
            resolved_cross_variants.insert(key, variant);
        }
    }
    resolved_cross_variants
}
```

Step-by-step walk-through:

Step	Code	Explanation
1	`for (func_name, model) in semantic_models`	Loop through every function's D1 model.
2	`for unresolved in &model.unresolved;_union_cases`	Only look at functions that Module 3 couldn't resolve.
3	`for other_func in &tu;_ast.functions`	Search ALL other functions.

Step	Code	Explanation
4	<code>`if other_func.name != *func_name`</code>	Skip the current function itself — look only in OTHER functions.
5	<code>`for transform in &other._model.local_union_transforms`</code>	Check Module 3's resolved transforms in the other function.
6	<code>`if transform.union_name == unresolved.union_name`</code>	Does this other function set the SAME union we're looking for?
7	<code>`if let Some(ref variant) = transform.local_variant_resolved`</code>	Does it have a resolved variant? <code>`ref`</code> borrows the string inside <code>`Some`</code> without moving it.
8	<code>`found_variant = Some(variant.clone())`</code>	Found it! Store the variant name.
9	<code>`break`</code> (both loops)	Stop searching once found.
10	<code>`resolved_cross_variants.insert("read_val:Data", "i")`</code>	Store the result: "for function <code>`read_val`</code> using union <code>`Data`</code> , the active variant is <code>`i`</code> ".

Understanding `if found_variant.is_some() { break; }`:

This breaks the OUTER loop (`for other_func`) after finding a result.

The `break` inside the innermost loop breaks the `for transform` loop.

We need this second break to exit the `for other_func` loop too.

Understanding `Some(ref variant)`:

- `ref` is needed here because `variant` is inside an `Option<String>` that we're borrowing.
- Without `ref`, Rust would try to MOVE the `String` out of the `Option` (which doesn't work with a `&` reference to the struct).
- `ref variant` means "give me a reference to the String inside `Some`".

The Unit Test

```
#[test]
fn test_cross_function_union_variant_tracking() {
    // set_val function: has local_union_transforms for "Data" -> variant "i"
    // read_val function: has unresolved_union_cases for "Data"

    let cross_resolutions = track_cross_function_union_variants(&tu_ast, &models);
    assert_eq!(
        cross_resolutions.get("read_val:Data"),
        Some(&"i".to_string())
    ); // Idiom 5 resolved!
}
```

File 6: `src/module4_interprocedural/data_dep_detector.rs`

(Idiom 6: Data-Dependent Runtime Ownership)

The Hardest Idiom — Runtime-Conditional Ownership: Sometimes whether memory is freed depends on a RUNTIME VALUE: `void conditional_free(int *p, int should_free) { p = malloc(10); if (should_free) { // only known at RUNTIME! free(p); } }` Rust requires ownership to be statically determined at compile time. If ownership depends on a runtime condition, we CANNOT directly translate this to safe Rust. Solution: Flag these variables and route them to: - `Option<Box<T>>` (the `ConservativeFallback` ownership kind) - Or the LLM repair agent in Module 6 for human-level reasoning

Lines 8-43: `detect_data_dependent_ownership()`

```
pub fn detect_data_dependent_ownership(
    tu_ast: &TranslationUnitAST,
    semantic_models: &HashMap<String, SemanticModel>,
) -> Vec<DataDepFlag> {
    let mut flags = Vec::new();

    for func in &tu_ast.functions {
        if let Some(model) = semantic_models.get(&func.name) {
            let lines: Vec<&str> = func.body_code.lines().collect();

            for (var_name, pattern) in &model.memory_patterns {
                if matches!(pattern, MemoryPattern::Free | MemoryPattern::Allocate) {
                    // Check if this malloc/free happens inside a conditional block
                    for line in &lines {
                        let trimmed = line.trim();
                        let is_conditional =
                            trimmed.contains("if (")
                            || trimmed.contains("if(")
                            || trimmed.contains("else if")
                            || trimmed.contains("switch");

                        if is_conditional
                            && (trimmed.contains(var_name) || func.body_code.contains("if"))
                        {
                            flags.push(DataDepFlag {
                                function_name: func.name.clone(),
                                variable_name: var_name.clone(),
                                reason: format!(
                                    "Ownership pattern '{:?}' for variable '{} ' is \
                                    conditioned on runtime control flow: '{}'",
                                    pattern, var_name, trimmed
                                ),
                            },
                        );
                        break; // One flag per variable per function
                    }
                }
            }
        }
    }

    flags
}
```

The Detection Logic Step-by-Step:

	Code	Explanation		
	<code>`for (var_name, pattern) in &model.memory::_patterns`</code>	Look at each variable's memory pattern from Module 3.		
	<code>`matches!(pattern, MemoryPattern::Free   MemoryPattern::Allocate)`</code>	MemoryPattern::Allocate`	Only interested in heap operations. `Free` and `Allocate` are the ownership-sensitive ones. `Read` and `Write` don't affect Rust ownership.	
	<code>`for line in &lines;`</code>	Scan every line of the function body.		
	<code>`is_conditional`</code>	Check if this line is a conditional branch: `if`, `if`, `else if`, or `switch`.		
	<code>`trimmed.contains(var_name)`</code>	Does this conditional line also involve our variable?		
	<code>`\`</code>	<code>`\`</code>	<code>func.body_code.contains("if")`</code>	Broader check if the function body has any conditional branches (not the safest approach — but safer).
	<code>`flags.push(DataDepFlag { ... })`</code>	Record the flag with details for the report.		
	<code>`break`</code>	Only one flag per variable per function — no need to check remaining lines.		

Understanding `matches!(pattern, MemoryPattern::Free | MemoryPattern::Allocate)`:

- `matches!` macro: returns `true` if `pattern` matches ANY of the listed patterns.
- `MemoryPattern::Free | MemoryPattern::Allocate` uses `|` as an OR pattern (not bitwise OR).
- Equivalent to: `pattern == MemoryPattern::Free || pattern == MemoryPattern::Allocate`.
- More concise and idiomatic in Rust.

Understanding the `reason` field:

```
reason: format!(
    "Ownership pattern '{:?}' for variable '{}{}' is conditioned on runtime control flow: '{}{}'",
    pattern, var_name, trimmed
),
```

- `{:?}` uses Debug format for `pattern` — prints `Free` or `Allocate`.
- This reason string appears in the Module 7 report, helping the developer understand WHY a variable was flagged.

The Unit Test

```
#[test]
fn test_data_dependent_ownership_detection() {
    // Function body: "int *p = malloc(10);\nif (should_free) {\n  free(p);\n}"
    // Variable p has MemoryPattern::Free
    // Function body contains "if"

    let flags = detect_data_dependent_ownership(&tu_ast, &models);
    assert_eq!(flags.len(), 1);
    assert_eq!(flags[0].variable_name, "p"); // Idiom 6 detected!
}
```

The Complete Module 4 Data Flow

Inputs:

```
tu_ast: &TranslationUnitAST          (Module 2 output)
semantic_models: &HashMap<String,    (Module 3 D1 output)
                  SemanticModel>

    |
    | Step 4.1: call_graph.rs
    v
summaries.call_graph = Graph { nodes: functions, edges: CallSites }

    |
    | Step 4.2: alias_analysis.rs
    v
summaries.alias_verdicts = {
  "main:foo:5" -> [AliasingVerdict { arg1: "&x", arg2: "&x", aliases: true }]
  ...
}

    |
    | Step 4.3: ownership_summary.rs
    v
summaries.ownership_transfer_summaries = {
  "free_buffer:buf" -> OwnershipTransferSummary { ConsumesOwnership }
  "alloc_buffer:out" -> OwnershipTransferSummary { CreatesOwnership }
  ...
}

    |
    | Step 4.4: union_tracking.rs
    v
summaries.cross_function_union_resolutions = {
  "read_val:Data" -> "i"
  ...
}

    |
    | Step 4.5: data_dep_detector.rs
    v
summaries.data_dependent_ownership_flags = [
  DataDepFlag { function: "conditional_free", variable: "p", reason: "..." }
  ...
]
```

Output: InterproceduralSummaries (D2) -- passed to Module 5

The 6 C Memory Idioms Taxonomy — Complete Reference

Idiom	Name	C Pattern	Detection Module	Rust Translation
1	Aliased &mut; params	<code>`foo(&x;, &x;)`</code>	<code>`alias_analysis.rs`</code>	Split borrows or use <code>`unsafe`</code>
2	T** consumes ownership	<code>`free(*buf)`</code>	<code>`ownership_summary.rs`</code>	<code>`Box`</code> parameter (callee drops)
3	T** creates ownership	<code>`*out = malloc(n)`</code>	<code>`ownership_summary.rs`</code>	Return <code>`Box`</code> or out-param
4	Overlapping pointer slices	<code>`combine(buf+2, buf+3, n)`</code>	<code>`alias_analysis.rs`</code>	<code>`split_at_mut`</code> or raw pointers
5	Cross-function union variant	<code>`set_val`</code> writes, <code>`read_val`</code> reads	<code>`union_tracking.rs`</code>	Rust enum with correct active variant
6	Data-dependent ownership	<code>`if (cond) free(p)`</code>	<code>`data_dep_detector.rs`</code>	<code>`Option`</code> fallback

Key Rust Concepts Introduced in This Module

	Example	Meaning		
	<code>`Graph`</code>	A directed graph from the petgraph crate.		
	<code>`let idx = graph.add_node(...)`</code>	Unique handle to a node in a petgraph graph.		
	<code>`graph.add_node("swap".to_string())`</code>	Adds a node, returns its NodeIndex.		
to,	<code>`graph.add_edge(a_idx, b_idx, call)`</code>	Adds a directed edge with associated data.		
	In tests	Built-in petgraph method.		
	<code>`for i in 0..args.len()`</code>	Range from 0 to n-1 (exclusive upper bound).		
	Pair iteration	Start inner loop AFTER outer index — avoids duplicate pairs.		
es(c) `	<code>`arg.trim_start_matches('&')`</code>	Remove a prefix character (only from start).		
	Pat2)	<code>`matches!(pattern, Free \</code>	<code>Allocate)`</code>	Test against efficiently.
	<code>`if let Some(ref variant) = ...`</code>	Borrow the inside of <code>`Some`</code> without moving it.		
	<code>`if found_variant.is_some() { break; }`</code>	Test if an <code>`Option`</code> contains a value without unwrapping.		
	Two <code>`break`</code> statements	Exit nested loops in sequence — inner <code>`break`</code> exits inner loop, outer <code>`break`</code> exits outer loop.		

Summary

Module 4 is the **interprocedural intelligence layer** of the C2SafeRust pipeline. It answers questions that require looking at MULTIPLE functions together:

4.1 Call Graph ([call_graph.rs](#)): Builds a petgraph directed graph of the whole program. Functions are nodes, call sites are edges. Enables "which functions call which?" queries.

4.2 Alias Analysis ([alias_analysis.rs](#)): For every call site, checks if any two arguments alias (point to the same memory). Detects **Idiom 1** (same variable passed twice) and **Idiom 4** (pointer arithmetic slices with the same base).

4.3 Ownership Transfer ([ownership_summary.rs](#)): For every T parameter, checks the function body for either `free(*param)` (Idiom 2 — **ConsumesOwnership**) or `*param = malloc(...)` (Idiom 3 — **CreatesOwnership**).

4.4 Union Tracking ([union_tracking.rs](#)): Takes unresolved union cases from Module 3 and searches OTHER functions for the active variant. Implements **Idiom 5** — cross-function reaching-definitions analysis.

4.5 Data-Dependent Ownership ([data_dep_detector.rs](#)): Scans for heap operations (malloc/free) inside `if/switch` blocks. Flags these as **Idiom 6** — ownership that cannot be statically determined. Routes to `Option<Box<T>>` fallback or LLM repair.

The complete [InterproceduralSummaries](#) (D2) is passed to Module 5 for ownership inference and code generation.

Document generated for C2SafeRust-LLM Mini Project S3

Module 4: Interprocedural Analysis — Complete Line-by-Line Explanation